



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

International Training & Cooperation Institute

Introduction to Blockchain Technology

Lesson 9 – Smart Contract Programming Fundamentals



Smart Contract Programming Fundamentals



- 1. What is a Smart Contract?**
- 2. Introduction to Solidity**
- 3. IDE for Smart Contract Development**
- 4. Ownership and Access Control**
- 5. Exercises**

1. What is a Smart Contract?

- ❖ A smart contract is a program stored on blockchain
- ❖ Automatically executed when called by a transaction
- ❖ Runs on Ethereum Virtual Machine (EVM)
- ❖ Execution result is the same on all nodes
- ❖ Cannot be changed after deployment

1. What is a Smart Contract?

Smart Contract vs Traditional Program

Traditional Program	Smart Contract
Runs on a server	Runs on blockchain
Can be updated	Immutable
Free execution	Costs gas
Trusted admin	Trustless execution

Q: If a bug happens in a smart contract, can we patch it?

1. What is a Smart Contract?

Smart contracts are programs deployed to Ethereum

- ❖ Written in **Solidity**
- ❖ Compiled to **EVM bytecode**
- ❖ Executed by all validators

Code must be correct, efficient, and secure.

Execution Flow

1. User sends transaction
2. Validator includes transaction in block
3. EVM executes contract code
4. Blockchain state is updated
5. Result is permanently stored

2. Introduction to Solidity

- ❖ Solidity is a high-level programming language
- ❖ Designed specifically for Ethereum
- ❖ Compiled into EVM bytecode

```
pragma solidity ^0.8.0;  
  
contract MyContract {  
  
}
```

Basic Solidity Program Structure

2. Introduction to Solidity

- ① **pragma**
 - Tells the compiler which Solidity version
- ② **contract:** like a **class**
 - Contains:
 - Data (state variables)
 - Functions
 - Deployed once → lives on blockchain
- ③ **State Variables**
 - Stored permanently on blockchain
 - Part of the contract's state
 - Common data types:
 - uint, string, bool
 - address
 - mapping
 - struct
 - array

```
pragma solidity ^0.8.0; ①

contract Hello {
    string public message; ③

    ④ constructor(string memory _msg) {
        message = _msg;
    }

    ⑤ function setMessage(string memory _msg)
    public {
        message = _msg;
    }

    ⑥ function getMessage() public view returns
    (string memory) { ⑦
        return message;
    }
}
```

2. Introduction to Solidity

4 constructor

- Runs only once
- Executes during deployment
- Used to initialize contract state

5 function

- Define contract behavior
- Can read or modify state
- Called via transactions or calls

6 Read-Only Functions: view

- Read blockchain state
- Do NOT modify data/cost gas

7 visibility

- Visibility of functions and state variables
- public/ private / internal / external

pragma solidity ^0.8.0;

contract Hello {
 string public message;

constructor(string memory _msg) {
 message = _msg;
}

function setMessage(string memory _msg)
public {
 message = _msg;
}

function getMessage() public view returns
(string memory) {
 return message;
}

3. IDE for Smart Contract Development

Remix

- ❖ Browser-based IDE
- ❖ No installation required
- ❖ Includes:
 - Solidity compiler
 - Ethereum test environment
 - Test accounts

3. IDE for Smart Contract Development

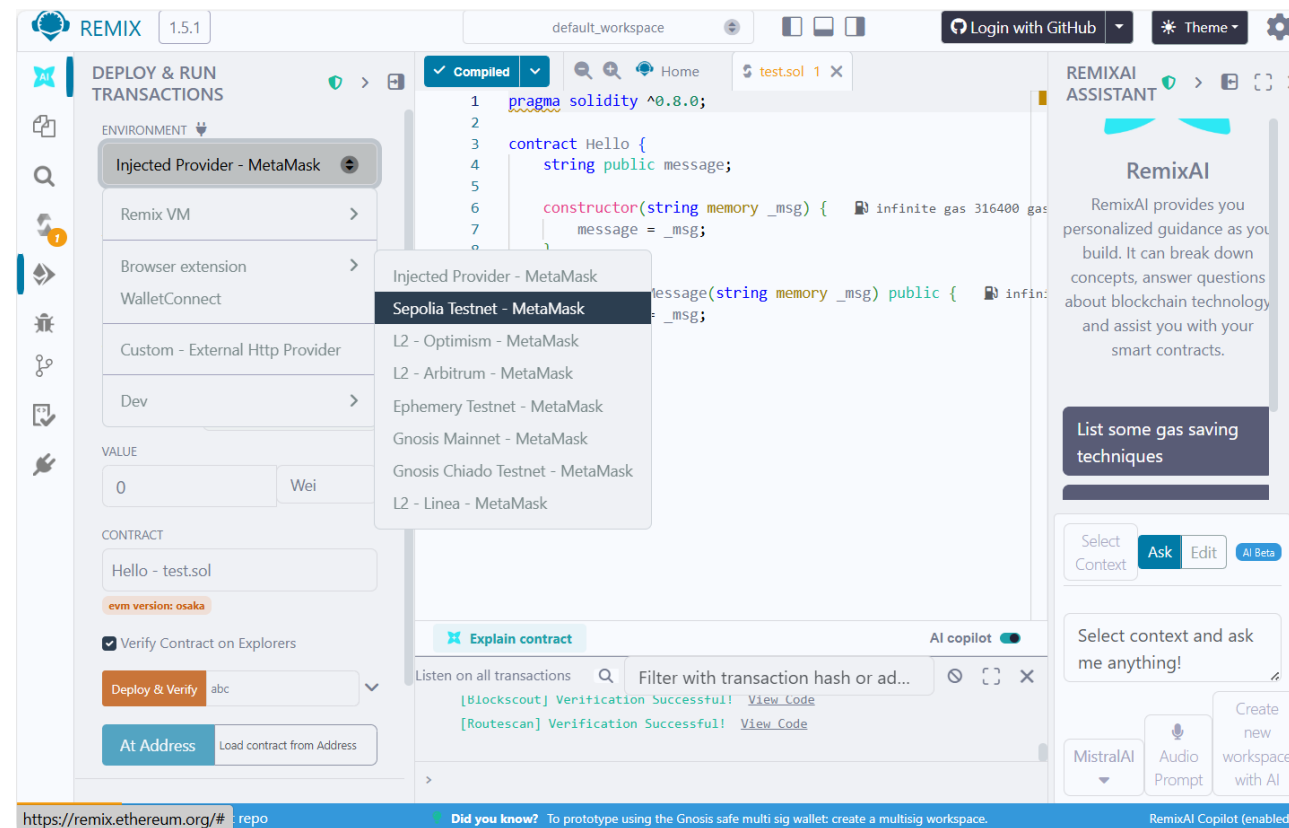
How to Deploy a Smart Contract

1. Open **remix.ethereum.org**
2. Create a new .sol file
3. Write contract code
4. Compile using Solidity compiler
5. Go to **Deploy & Run**
 - For a quick test, set **Environment** to **Remix VM** (Osaka/Shanghai)
6. Click **Deploy**

3. IDE for Smart Contract Development

Connect Remix to MetaMask

- ❖ Open remix.ethereum.org.
- ❖ Go to **Deploy & Run Transactions** panel.
- ❖ Environment: Select **Sepolia Provider - MetaMask**.
- ❖ MetaMask pops up => confirm connection.



3. IDE for Smart Contract Development

Task: Deploy the sample contract above using Remix

- ❖ Change the initial message
- ❖ Call *setMessage()*

Questions:

1. Which function costs gas?
2. Which one does not?
3. Why?

```
pragma solidity ^0.8.0;

contract Hello {
    string public message;

    constructor(string memory _msg) {
        message = _msg;
    }

    function setMessage(string memory _msg)
    public {
        message = _msg;
    }

    function getMessage() public view returns
    (string memory) {
        return message;
    }
}
```

4. Ownership and Access Control

Ownership: Allow only a designated owner can perform sensitive actions.
Solidity's global context variables:

- ❖ **msg.sender**: the address of whoever calls the function
- ❖ Blockchain contracts are **public**
- ❖ Anyone can call public functions
- ❖ Access control must be coded explicitly
- ❖ Ownership is a common access-control pattern

```
address public owner;  
  
constructor() {  
    owner = msg.sender;  
}
```

Example:

```
function setMessage(string memory _msg) public {  
    require(msg.sender == owner, "Not owner");  
    message = _msg;  
}
```

Q: What happens if we remove the ownership check from the **setMessage** function?

5. Exercises

Ex 1: Write and deploy the **MyProfile** Contract Store:

- Your Name
- Student ID
- Class

Feature:

- Update info
- Read info

Only creator can update data
Anyone can read data

5. Exercises

Ex 2: Build a **StudentRegistry** contract

Store:

- Student Name
- Student ID
- Class

Features:

- ❖ Add student (Student name/Student ID/Class)
- ❖ Update student (Update student name/class, given a student ID)
- ❖ Read student info (Given a student ID)

Only creator can update data

Anyone can read data

5. Exercises

Ex 3: Build a **StudentRegistryAlt** contract
Store:

- Student Name
- Student ID

Features:

- ❖ Add student (Student name/Student ID)
- ❖ Update student (Update student name, given a student ID)
- ❖ Read student info (Given a student ID)

Only creator can update data

Anyone can read data

Compare gas fee with the contract in **Ex 2**

Hint: Check gas fee in Remix Log

